

29. Strava (Activity & Fitness Tracking)

Interviewer: Let's design Strava. Someone goes for a run or a bike ride with their phone or GPS watch. When they're done, the app uploads the activity: a map of where they went, their pace, elevation gained — and if their route crossed any of the millions of community-defined "segments" (a specific stretch of road or trail), we rank their time against everyone else who's ever ridden that same stretch, on a **leaderboard**. Plus a social feed of your friends' activities. Design the backend.

Candidate: Great one — this isn't a "hot key under a thundering herd" problem like a flash sale. It's a **data-shape and fan-out** problem: every activity is a fat, ordered sequence of GPS points that needs to become durable storage, cheap stats, and — the hard part — a spatial match against a huge, mostly-static universe of segments, whose result gets read far more often than it's computed. Let me clarify first.

1. Clarifying the requirements

Candidate: A few questions: 1. What exactly is a "segment" — is it a fixed, pre-defined polyline (a stretch of a specific road/trail), and do we need to detect that an activity *traversed* it, in the right direction, not just passed nearby? 2. How fresh does the leaderboard need to be — the instant the activity finishes uploading, or is "within a minute or two" fine? 3. Privacy — can users hide the exact location of, say, their house? 4. Scale — roughly how many athletes, activities/day, and how many segments exist globally?

Interviewer: Yes — a segment is a fixed polyline (start/end + path), and matching means the activity's track followed that path, start to finish, in order. A minute or two of leaderboard delay is totally fine — nobody expects instant fame. Users can define a **privacy zone** (e.g., a radius around home) that's scrubbed from anything public. Assume **100M athletes**, **10M activities uploaded per day**, and **~40M segments** worldwide.

Candidate: Then here's what I'm building to.

Functional - Upload a GPS activity (points: lat/lon/elevation/timestamp, ± heart rate/cadence). - Compute derived stats: distance, moving time, pace, elevation gain/loss. - Match the activity against the segment universe; for each match, record an **effort** (elapsed time) and update that segment's **leaderboard** (ranked, filterable by time window / gender / following). - Social feed of followed athletes' activities, with kudos/comments. - Heatmap of where people move (aggregated, privacy-respecting). - Privacy zones that redact the public view of an activity's start/end area.

Non-functional - **Durability of raw data first** — GPS tracks are irreplaceable; never lose them. - **Read-heavy, not write-spiky**. Uploads are large but comparatively low-QPS; leaderboard and feed *reads* are the real volume, and they happen over and over for data that rarely changes. - **Segment matching must scale sub-linearly** in the number of segments — we cannot geometrically compare every activity against 40M segments. - **Freshness**: leaderboards/feed can be eventually consistent (seconds-to-minutes); a

user's own activity page should reflect their own upload immediately (basic stats, at least). - **Privacy is a hard requirement**, not a nice-to-have.

2. Estimation — the shape of the data

Candidate: The numbers here point away from "hot lock" and toward "storage + fan-out."

100M athletes, 10M activities uploaded/day.

Points per activity: a 45-min run @ 1 pt/2s $\approx$ 1,300 pts; a 3-hour ride @ 1 pt/1s $\approx$ 10,800 pts. Call the average $\sim$ 2,000 points/activity.

points/day = $10M * 2,000$ = 20 BILLION points/day
raw file $\approx$ 300 KB/activity (FIT/GPX-style encoding)
raw storage = $10M * 300KB$ $\approx$ 3 TB/day of raw files

If we stored one relational ROW per point (naive):

20B rows/day * $\sim$ 40 bytes/point $\approx$ 800 GB/day just for point rows
→ this alone rules out "one row per GPS point in Postgres."

Upload QPS is a soft spike (weekend mornings), not a stampede:

$\sim$ 30% of daily activities land in a 3-hour window
 $3M / 10,800s \approx 280/s$ average, maybe 1,000–1,500/s at peak
→ nothing like the 670k/s giveaway problem. Uploads are BIG, not FAST.

Segments: $\sim$ 40M worldwide. A single activity's track, after spatial filtering (§6), only ever needs to be geometrically checked against a few dozen to a few hundred CANDIDATE segments – never all 40M.

Reads dwarf writes: feed scrolling + leaderboard views $\approx$ 200M page views/day
 $\approx$ 2,300/s average, bursting to $\sim$ 10k/s in the evening.

⇒ The core challenge isn't lock contention. It's: (a) store a firehose of GPS points cheaply and durably, (b) turn each new track into "which of 40M segments did this touch" without a linear scan, and (c) make sure the RESULT of that work is read millions of times more often than it's computed.

3. A simple V1

Candidate: Let me state the naive version so we can see exactly where it buckles.

```
[ App ] --upload GPX/FIT--> [ Web server ]
                             | parse file, sum haversine distances,
                             | sum elevation deltas -> stats
                             v
                             [ Single Postgres ]
                             activities(id, athlete_id, distance, points_json, ...)
```

One server parses the file inline, computes stats synchronously, stuffs the entire point array into a JSON/array column, and returns the activity page. No segments, no leaderboard, no feed, no heatmap

yet.

Why it breaks under real load: - Storing a 2,000-point array **inside** a relational row bloats the table; every backup, replication stream, and `SELECT *` that touches `activities` now drags megabytes of point data it usually doesn't need (the summary is all the activity *feed* wants). - Parsing + stats computation on the **request path** ties up a web worker for however long a 10,800-point file takes to parse — fine at low volume, a latency and throughput problem at 10M/day. - There is **no way to do segment matching** without, for every upload, comparing the track against some subset of 40M segments — doing that inline would make every upload as slow as the slowest geometric search across the whole segment universe.

4. Scaling up: the write flood, the read storm, and multi-region

Interviewer: Okay, now make it real. 10M uploads/day, way more reads, and this is a global product.

Candidate: Three moves, in the order the interviewer usually pushes on them:

More users → get parsing off the request path. The client uploads the raw file **directly to object storage** via a pre-signed URL; the API only writes a lightweight `activities` row (`status = pending`) and enqueues a `activity-uploaded` event. A worker pool does parsing, stats, and segment matching **asynchronously**. Upload responds in tens of milliseconds regardless of file size.

Read path → this is where the real volume lives. Feed and leaderboard reads are ~1000x the upload rate. Cache activity summaries and rendered map images behind a CDN; keep segment leaderboards in **Redis sorted sets** (built for exactly this: ranked reads, cheap inserts — detailed in §6); fan out new activities to followers' cached feed lists at write time (most athletes have modest follower counts, so fan-out-on-write is cheap; a rare "celebrity" athlete with millions of followers falls back to fan-out-on-read, merged at query time).

Write path → stop treating one Postgres instance as the answer for everything. Split concerns by access pattern instead of forcing one store to do all of it: raw files in object storage, the giant point stream in a horizontally-scalable store keyed by activity, small structured "social graph + activity summary" rows in a relational store that actually needs joins and ACID. (Full reasoning in §5.)

Multi-region. Athletes and segments are geographically local — a Parisian's ride only ever matches Parisian segments. So: shard the spatial segment index and the Redis leaderboard clusters by **region/geohash prefix**, so matching work happens near the data it needs. Raw files and rendered map tiles replicate to a CDN close to where they're viewed. A segment that happens to straddle a regional boundary is rare enough to just "home" it in one region and treat the other side as a fallback lookup.

5. API + data model

```
POST /uploads                -> { activityId, uploadUrl } (pre-signed, direct-to-S3)
POST /activities/{id}/complete -> enqueues processing; activity.status = "processing"
GET  /activities/{id}        -> summary + map polyline + matched segments (may lag)
GET  /segments/{id}/leaderboard?scope=overall|year&gender=&following=
GET  /feed?cursor=
GET  /athletes/{id}/heatmap  -> pre-rendered tile URLs
```

Data model, split by access pattern, not by "one true database":

Store	Table / key	Why here
Object storage (S3)	raw FIT/GPX file per activity	Immutable blob, cheap, durable, never needs a query — just a fetch by key.
Relational (Postgres)	activities(id, athlete_id, sport, start_time, distance_m, moving_time_s, elevation_gain_m, start_latlng, status)	Small rows, needs joins with athletes/kudos/comments/follows, ACID matters for the social graph.
Wide-column / KV (Cassandra or DynamoDB)	tracks(activity_id) -> compressed polyline blob (points, timestamps, elevation)	Access pattern is always "give me the <i>whole</i> track by activity id," never a time-range slice — a plain partition-by-key store fits and scales writes horizontally.
Wide-column, partitioned by segment_id	segment_efforts(segment_id, athlete_id, activity_id, elapsed_time_s, ts) — unique on (segment_id, activity_id)	Huge append volume (every matched segment × every activity); partitioning by segment keeps "all efforts for segment X" cheap to scan for rebuilds.
Redis	lb:{segmentId}:{scope} sorted set, score = elapsed_time_s, member = athlete_id	The <i>read-optimized</i> ranked view derived from segment_efforts — see §6.
Relational	segments(id, name, polyline, bbox, grid_cells[], distance_m, elevation_profile)	Reference data, low write volume, needs to be joined/looked up by admins and by the matching pipeline.

Why not put the point stream in the same relational DB as activities ? Two different shapes and access patterns: `activities` is small, joined, transactional rows (social/ACID territory); the point stream is a huge, single-key blob you always read as a whole and never join against. Forcing both into Postgres means every table scan, backup, and replication stream on the *small* table is dragging the *huge* one along for the ride.

Why not a "real" time-series database (InfluxDB/TimescaleDB) for the track points? Those shine when you query **ranges across many parallel series** — "average CPU over the last hour across 10,000 hosts." Our access pattern is the opposite: fetch **one entire track by activity id**, never a time slice across many activities' series at once. A wide-column/KV store keyed by `activity_id` gives us that for less operational cost than a TSDB's downsampling/retention machinery we'd never use.

Privacy zones are applied at the **boundary between internal and public data**: the true, unmodified track is always used for stats and segment matching (matching needs the real path), but any point inside a saved private radius is stripped/fuzzed **before** it's rendered on a public map, included in a heatmap

tile, or returned from a public API. Sanitize once, at every outbound edge — not by trusting every downstream consumer to remember to do it.

6. The segment-matching & leaderboard pipeline

Interviewer: Walk me through what actually happens between "upload finishes" and "my effort shows up on a leaderboard."

```
[activity-uploaded event] (Kafka, partitioned by athlete_id)
    ↓
[ Parsing worker ]  parse raw file -> stats (distance, elevation, pace) -> write activities ro
    ↓
[ Segment-matching worker ]
    | 1. compute which GRID CELLS the track passes through (§11 has the detail)
    | 2. look up candidate segments for those cells (inverted index: cell -> segment_ids)
    | 3. for each candidate, run the precise geometric match (corridor + direction)
    | 4. for each real match: elapsed_time = exit_ts - enter_ts
    ↓
[ Write segment_effort ]  upsert, unique on (segment_id, activity_id) -> idempotent on retry
    ↓
[ ZADD lb:{segmentId}:{scope} elapsed_time athlete_id ]  (Redis sorted set, several scopes)
    ↓
[ ZRANK check ] -> if new personal best or new #1 -> emit "PR" / "KOM" notification event
```

This is the whole point of **precomputing at write time**: the expensive geometric work happens once, off the user's critical path, in a worker; every future leaderboard *view* is just an $O(\log N)$ Redis lookup, no matter how many times it's viewed. Deep-dive A makes the case for why the opposite (compute at read time) doesn't work here.

7. Consistency, async, and caching

- **Strong / read-your-own-write:** an athlete's own activity summary (distance, stats) is visible to them the moment parsing finishes — that's a simple, fast, synchronous-feeling step even though it runs in a worker, because it doesn't depend on segment matching.
- **Eventual (seconds to a couple minutes):** segment efforts, leaderboard position, feed entries for followers, heatmap tiles. Nobody expects to see their KOM crown the millisecond they cross the line; "soon" is the actual product requirement (confirmed in §1).
- **Exactly-once where it matters:** `segment_efforts` upserts are idempotent on `(segment_id, activity_id)`, so an at-least-once queue retry after a worker crash never double-writes a leaderboard entry.
- **Async, off the critical path:** parsing, matching, feed fan-out, PR/KOM notifications, and heatmap aggregation are all queue-driven workers, not request-time work.

- **Heatmaps are a batch job, not a live query.** A nightly/weekly Spark job aggregates public, privacy-zone-respecting points into tile-level counts, rendered as vector tiles in S3 behind a CDN. Computing a heatmap live, per map viewport, from billions of raw points would be far too slow for something that changes slowly anyway.
- **Caching:** CDN caches rendered map images and static tile sets; Redis is the cache for leaderboards (no separate cache layer needed — the sorted set is both the store and the fast path); a very popular segment's top-10 view can additionally be cached with a short TTL, since "am I #1,204 or #1,206" doesn't need to be exact on every page load, while a user's own ZRANK lookup stays a live, cheap, single-key read.

8. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Raw GPS file	Object storage (S3)	Cheap, durable, immutable blob fetched by key. <i>Not</i> a DB blob column — that bloats backups/replication for data you only ever fetch whole.
Track point stream	Wide-column/KV (Cassandra/DynamoDB), partitioned by activity_id	Matches the access pattern ("whole track by id"), scales writes horizontally for 20B points/day. <i>Not</i> a time-series DB — that's built for range queries across many parallel series, which we never do.
Activity/social metadata	PostgreSQL	Needs joins (kudos, comments, follows) and ACID; row volume (10M/day) is trivial once points aren't inline.
Segment reference data + inverted grid index	Relational + geospatial index (PostGIS / S2 cells)	Segments are low-write, need spatial queries ("which segments touch this cell"); a proper spatial index turns 40M segments into a handful of candidates per lookup (see §11). <i>Not</i> a linear scan of all segments per upload — computationally impossible at this scale.
Segment leaderboards	Redis sorted sets	O(log N) ranked insert and range reads, native top-K and rank lookup. <i>Not</i> ORDER BY elapsed_time LIMIT 10 on a relational table per view — that's a sort/index-scan under the highest-QPS read path in the whole system.
Matching + fan-out pipeline	Kafka + async worker pools	Durable, replayable, ordered per partition; segment matching and feed fan-out are CPU/IO work that must never sit on the upload request path.
Heatmaps	Offline batch (Spark/Flink) → static tiles on CDN	Aggregating billions of points per map view live is far too slow for data that only needs to refresh daily/weekly.
Privacy zones	Sanitize once, at every outbound boundary	Keeps the <i>internal</i> matching pipeline accurate (real coordinates) while guaranteeing no downstream consumer (map, heatmap, public API) can leak a redacted point by forgetting to filter it themselves.

Say-it-out-loud justification: "Raw files go to object storage because they're big and immutable; the point stream lives in a wide-column store keyed by activity because we always read it whole, never as a time range; segment matching runs asynchronously against a spatial index so we never geometrically compare a track to more than a handful of real candidate segments; and Redis sorted sets hold the leaderboard because it's the single highest-read-QPS piece of this system and needs $O(\log N)$ ranked lookups, not a re-sort on every page view."

9. Failure modes & graceful degradation

Failure	What happens	Mitigation
Segment-matching worker crashes mid-batch	Retry via at-least-once queue delivery	Upsert on <code>(segment_id, activity_id)</code> — retries never double-count an effort
Redis leaderboard node down	Rank queries on that segment fail/stale	Replica failover; worst case, rebuild the sorted set from the durable <code>segment_efforts</code> table, which is the real source of truth
Object storage write fails mid-upload	Raw file missing	Client retains the local recording until the server acks a successful upload; retried with backoff
Weekend-morning upload spike exceeds worker capacity	Processing backlog grows	Queue absorbs it; activity shows <code>status = processing</code> ; upload itself was already accepted (raw file safely stored)
GPS noise (tunnel loss, device glitch)	Track has gaps/false points	Smoothing/outlier filtering at parse time; a segment that can't be confidently matched is simply skipped, not force-matched
Segment definition changes (route re-drawn)	Old efforts may no longer be valid	Versioned segment + a backfill job re-runs matching only for activities whose grid cells overlap the new polyline

10. Follow-up curveballs

Interviewer: A flaky network causes the app to retry an upload. How do you avoid two copies of the same run?

Candidate: The client generates a stable local recording ID (a UUID created when the recording starts) and sends it as an idempotency key. `activities` has a unique constraint on `(athlete_id, client_recording_id)`; a retried upload just returns the existing activity.

Interviewer: Two athletes post the *exact* same elapsed time on a segment. How do you rank them?

Candidate: Redis sorted sets break ties by **member ordering** when scores are equal, but I want a deterministic, meaningful tiebreak — so I make the score a composite: `elapsed_time_s * 1000 +`

`tiny_offset_from(activity_id or effort_ts)` , or store elapsed time to millisecond precision. Practically: earlier `effort_ts` wins the tie, applied consistently.

Interviewer: Someone deletes the activity that currently holds the segment's #1 spot.

Candidate: `ZREM` the leaderboard entry and soft-delete the `segment_effort` row (keep it for audit/undo). The next-ranked athlete is now #1 automatically — sorted sets re-rank for free. Separately, queue a job to recompute that athlete's *own* personal-best for that segment from their remaining efforts, since their PR might have just changed too.

Interviewer: We just added a brand-new segment along a popular greenway. Millions of past rides already went down that path — how do you populate its leaderboard retroactively?

Candidate: Run the *same* matching pipeline as new uploads, but as a **backfill batch job** scoped by the spatial index: look up which past activities have tracks touching the new segment's grid cells (the inverted cell index works both directions), and only geometrically match those — not the entire history. Same precompute-once philosophy, just triggered by a new segment instead of a new activity. This is exactly the reasoning behind Deep-dive A, next.

11. Deep-dive: "Why not compute segment matches and leaderboards on the fly, per request?"

Interviewer: All this async matching infrastructure — grid indexes, workers, sorted sets — feels like a lot. Why not just store the raw tracks, and when someone opens a segment's leaderboard page, run the geometric match against every activity that could plausibly have covered it, right then, and sort the results? Simpler system, one source of truth.

Candidate: It's the classic **read-vs-write amplification** question, and here the numbers land hard on one side. An activity is created **once**. Its segment matches and leaderboard position are then **viewed** by the athlete themselves, their followers, competitors checking the leaderboard, and anyone browsing that segment — potentially thousands of times over the segment's lifetime. Computing the same expensive geometric result on every single one of those views is paying the same bill over and over for an answer that hasn't changed since the ride finished.

compute-on-read:

```
view #1 -> full geometric match
view #2 -> full geometric match
view #3 -> full geometric match
... (repeat forever, same answer)
view #10,000 -> full geometric match
```

precompute-on-write:

```
upload -> ONE geometric match,
          write the result
view #1 -> O(log N) sorted-set read
view #2 -> O(log N) sorted-set read
... (cheap, forever)
```

Concern	Compute-on-read	Precompute-on-write (chosen)
Leaderboard page latency	Must re-run matching/sorting across every plausibly-overlapping activity on every view	$O(\log N)$ sorted-set range read, regardless of view count
Cost under a popular segment (thousands of views)	Cost scales with <i>views</i> — the more popular the segment, the more redundant recomputation	Cost scales with <i>uploads</i> , computed once; popularity is free
Cost for a segment nobody ever checks	Zero — never computed if never viewed	A small amount of "wasted" upfront work for efforts nobody looks at (cheap; matching is bounded by candidate count, not by popularity)
Consistency when the segment definition changes	Always fresh by construction — recomputed every read	Needs an explicit backfill job (\$10) to catch up historical activities
System complexity	Looks simpler (no pipeline/index to maintain) — until you realize you still need the spatial index just to avoid scanning all activities per view	Needs a pipeline + spatial index, but the read path stays trivially simple and fast

The one place compute-on-read genuinely wins is the "always fresh" row — but we established in §1 that a minute or two of leaderboard staleness is an explicit, acceptable requirement, so that advantage doesn't matter here. And even compute-on-read still needs the same spatial index to avoid scanning every activity per view — it just moves the cost from "once, async" to "every time, on the request path."

Rule of thumb: *When a computed result is read far more often than its input changes, precompute it once and store the derived fact. Reserve compute-on-read for results that are rarely viewed or whose correctness can't tolerate any staleness — and even then, you'll still need whatever index keeps you from scanning everything per request.*

12. Deep-dive: segment matching at scale — millions of segments, one new track

Interviewer: Make this concrete. A new activity comes in with a few thousand GPS points. There are 40 million segments in the world. How do you find which ones it actually traversed — correctly, and fast — plus update the leaderboard?

Candidate: This has to be sub-linear in the number of segments, or it never scales, no matter how many workers we throw at it. I'll build it in two phases: a cheap spatial prefilter, then an expensive-but-rare precise geometric check only on real candidates.

Phase 1 — coarse spatial filtering (grid / cell index)

Tile the world into cells (e.g. Google's S2 cells or geohash-style tiles, roughly 100-200m across).

Precompute once, for every one of the 40M segments, which cells its polyline passes through, and store the **inverted** index: `cell -> [segment_ids]`.

Precomputed inverted index (built once per segment, rebuilt only if a segment's path changes):

```
cell(48.86,2.35) -> [S102, S104, S190, S233]    <- e.g. a dense park area
cell(48.86,2.36) -> [S104, S190, S502]
cell(45.50,-73.6) -> [S9981]
... (40M segments spread across the world's cells)
```

New activity's track walked cell-by-cell:

```
track touches cells: { (48.86,2.35), (48.86,2.36), (45.50,-73.6) is NOT touched }
```

```
candidate segments = UNION of the cell lists the track actually touched
                    = { S102, S104, S190, S233, S502 }
                    -- dozens, not 40 million --
```

Phase 2 — precise geometric matching (only on the handful of candidates)

For each candidate, buffer the segment's polyline by a tolerance (e.g. $\pm 25\text{m}$) to form a **corridor**. Check whether the track has a contiguous, correctly-ordered run of points inside that corridor, entering near the segment's start and exiting near its finish, in the right direction.

Segment S104's corridor (buffered polyline):

```
start o=====o finish
      | ~50m-wide corridor |
```

Track A — actually rode it:

```
.....o-----follows the corridor start to finish-----o.....
      ^ enters, correct direction                               exits ^
=> MATCH. elapsed = ts(exit) - ts(enter) -> write segment_effort, ZADD leaderboard
```

Track B — clipped the same grid CELL but never entered the corridor
(e.g. a parallel street one block over):

```
.....o.....
      stays outside the buffer the whole time
=> NOT a match — discarded before it ever touches a leaderboard write
```

The cell lookup in Phase 1 only narrows candidates down to "plausibly nearby" — Phase 2's corridor-and-direction check is what actually prevents false positives like Track B.

Ranked approaches

1. **Full scan against all 40M segments (rejected).** A geometric containment check per segment, per upload, is simply not computable at this volume — not even close.
2. **Bounding-box prefilter only (partial credit).** Filter using each segment's/track's overall bounding box. Better than nothing, but a long, winding ride's bbox can be huge (or a route that loops back near itself), so it still admits far too many false candidates — especially in dense cities where thousands of segments share one bounding region.

3. **Grid/cell inverted index (recommended).** Prune by the cells the track *actually passed through*, not just its overall bbox — tight candidate sets even for long, winding routes. Use **variable cell resolution** (finer cells in dense areas like a city park with thousands of overlapping segments, coarser cells in open countryside) so the candidate count stays bounded everywhere, not just in the average case.
4. **Two-phase refinement, paired with #3 (recommended).** Before running the (relatively) expensive corridor-and-direction check, do one more cheap pass — a per-candidate bbox and rough-heading comparison — so the heavy geometry only ever runs against a truly small, plausible set, even after cell lookup returns a few hundred candidates in a segment-dense area.
5. **Direction-aware corridor matching as the final gate (mandatory regardless of the above).** Without checking direction and contiguous order, a track that merely grazes a segment's cell (Track B above) would false-positive onto the leaderboard.

Recommended: **#3 + #4 + #5 together** — grid index to prune from 40M to dozens/hundreds, cheap heading/bbox refinement to shrink further in dense areas, then the precise corridor check as the last, authoritative gate before anything is written.

Once a match is confirmed: write `segment_effort` (idempotent upsert), `ZADD` the relevant leaderboard sorted sets (overall, this-year, gender, following — a small fixed set of scopes per segment), and compare against the athlete's prior best / the segment's current #1 to fire a PR/KOM notification. Because each leaderboard is its own Redis key, even a viral, wildly popular segment (a famous climb) doesn't create a hot key that affects any *other* segment — and a short-TTL cache on that one segment's top-10 view absorbs read pressure if it ever needs it, without touching the write path at all.

What to say out loud: *"The spatial index does the heavy lifting — it turns 'compare against 40 million segments' into 'compare against a few dozen real candidates' by indexing segments into the same grid cells a track can touch. Direction-aware corridor matching is what actually gates correctness, preventing a route that merely brushes a segment's area from false-positiving onto a leaderboard. And because matching runs asynchronously and writes a durable effort record, every leaderboard view afterward is just a cheap sorted-set read, no matter how many times — or how many candidates — it took to get there."*

Summary — the mental model

Strava is not a hot-lock contention problem — it's a **data-shape and fan-out** problem: turn a firehose of GPS points into durable, cheap-to-query storage; **precompute** the expensive geometric work (segment matching) once, asynchronously, at upload time, because that result gets read orders of magnitude more often than it's computed; and use a **spatial grid index** to make "which of 40 million segments did this track touch" sub-linear instead of impossible. Object storage holds the irreplaceable raw files, a wide-column store holds the point stream keyed by activity, Postgres holds the small relational social graph, and Redis sorted sets serve the single highest-read-QPS surface — the leaderboard — at $O(\log N)$ per lookup, forever, no matter how many people check it.